

PROJECT

ONDOL

이진철 / 원루빈 / 박범진

(IT개발 3팀 & IT운영팀
- ex. 클라우드플랫폼팀)

TEXT-TO-SQL DATA REPORT AGENT

자연어로 묻고,
SQL은 맡기고,
자연어로 받는다.

Text-to-SQL

RBAC + SQL Guard

자연어 리포트

오늘 보여드릴 것

문제 정의 → 솔루션 구조 → 데모 → MetIQ와의 연결

01

배경

왜 Text-to-SQL인가

현업의 문제와 기존 도구의 한계

02

솔루션

Agentic 파이프라인

자연어 질문 → SQL → DB → 자연어 리포트

03

데모

화면으로 보는 ONDOL

채팅 · 관리자 · BI · DB/권한

04

로드맵

MetIQ 통합 비전

사내 AI 플랫폼으로 도약

"SQL을 대신 써주는 도구가 아니라, 권한과 의미를 지키며 의사결정을 돕는 데이터 에이전트."

"이번 달 클라우드 비용, 프로젝트별로 좀 정리해 줄래요?"

한 줄 질문이 유관 부서에 도착하면, 실제로는 이런 일이 벌어집니다.

STEP 01

요청자가
자연어(메일 등)로 묻는다

"이번 달 비용", "최근 위험률"
— 맥락 이해는 그런 범위인 것이다

STEP 02

담당자가 테이블 확인하고
SQL을 짤다

테이블을 찾고, 조건 확인하고,
필터 맞추고, 검증하고...

STEP 03

SQL 결과를 전달하고
해석해준다.

"이게 무슨 뜻이죠?"
다시 회의, 다시 질문

결과

의사결정
3일 뒤 도착

담당자 바쁘면
X2배는 기본...

SQL을 쓸 줄 모르면

데이터에 접근하는 데
n명의 사람을 거쳐야 한다

SQL을 쓸 줄 알아도

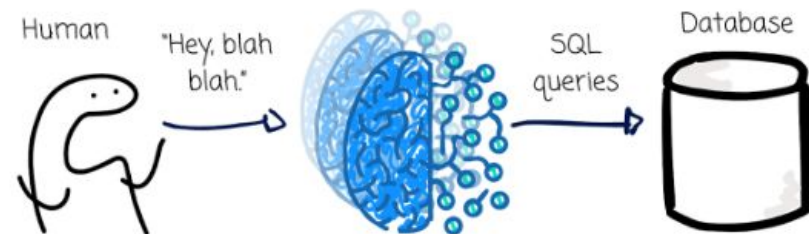
"이번 달", "위험률" 같은
업무 표현은 매번 다시 해석

그리고 가장 큰 문제

담당자는 이런 요청 하나하나 다
처리할 시간이 없어요!!!

왜 지금, Text-to-SQL인가

대시보드는 미리 만든 질문만 답한다. 유관 부서들은 모든 질문을 받아내지 못한다.
자연어 → SQL → 자연어 리포트가 그 사이를 메운다.



대시보드의 한계

정해진 질문만

SQL의 한계

잘 줄 아는 사람만

Text-to-SQL의 약속

자연어로 묻고 자연어로 받는다

☁ 클라우드 비용

"이번 달 비용 프로젝트별로
정리해 줘"

⚠ 장애 운영

"대형팝 장애가 가장 많은 팀은?"

🔒 보안 알림

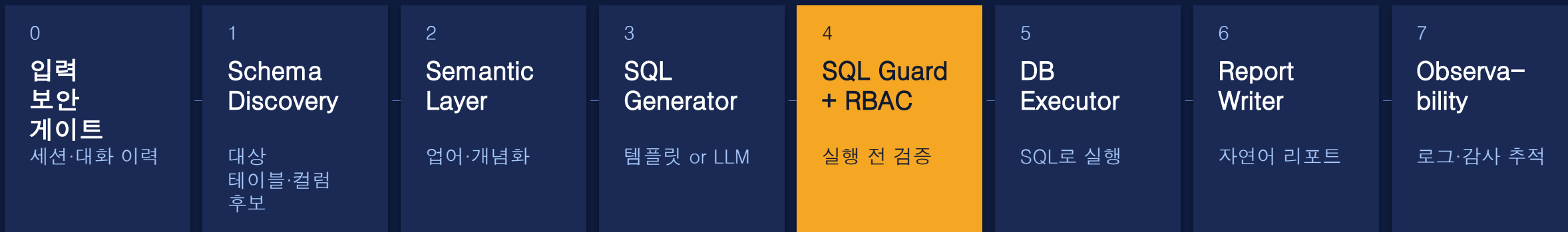
"최근 위험률, 부서별 담당 설명"

💰 자원 낭비

"거의 안 쓰는데 비용 나가는
서비스는?"

한 문장이 리포트가 되기까지

8개의 핵심 분리 노드 — 한 번의 LLM 호출이 아니라, 각 단계가 자기 일만 한다.



01

사용자는 자연어로 쉽게 이해한다.

SQL, 스키마, 권한 정책도 확인이 가능하다.
사용자는 자연어로 생성된 리포트와
AI가 리포트를 생성한 모든 과정을 볼 수 있다.
사용된 테이블과 SQL 쿼리까지도!

02

시스템은 각 단계를 본다

스키마 확인 → 의미 적용 → SQL 생성
→ 검증 → 실행 → 리포트, 단계마다 책임이
분리된다. 분업이 철저한 Agentic한 설계!

03

운영자는 흔적을 본다

비용, 감사 로그, DB/권한 매트릭스, 차단된
요청까지 — 모두 화면에서 확인 가능.

(혁신금융서비스 실사 요건 참고하여 반영함)

사용자 화면 속 5단계 처리

예시 질문

"이번 달 클라우드 비용을
프로젝트별로 리포트로
정리해 줘"

사용자가 보는 답

자연어 리포트

이번 달 클라우드 비용은 총 \$184,200
이며, 그 중
Insurance Core 프로젝트가 38%로 가장
큰 비중을 차지합니다...

▶ 근거 데이터와 SQL 보기

UI에서 실시간으로 표시되는 단계

1

데이터 구조 확인

관련 테이블 · 컬럼 · 데이터 후보를 먼저 본다 — 없는 컬럼을 상상하지 않기 위해

2

업무 의미 적용

"이번 달", "프로젝트별", "비용" 같은 표현을 Semantic Layer가 데이터 조건으로 번역

3

SQL 생성

프리셋이 있으면 검증된 템플릿 사용, 없으면 gpt-4.1-nano로 SELECT 생성

4

SQL 검증 + RBAC + DB 조회

SELECT-only, 실제 테이블 필수, 역할별 권한 확인 — 실행 전 차단

5

최종 자연어 리포트 작성

결과값 기반 요약 · 상세 해석 · 다음 확인 포인트 — SQL은 접한 영역에만 보존

REPORT READY

"이번 달"은 DB에 없다

사람이 쓰는 업무 표현과, DB가 이해하는 컬럼·조건 사이의 번역

사용자 표현	Semantic 해석	SQL 조건
"최근"	최근 30일 또는 최신 month	<code>submitted_at ≥ date('now', '-30 days')</code>
"위험률"	High risk 요청 비율	<code>SUM(risk_level='High') / COUNT(*)</code>
"클라우드 비용"	비용 예측/실적 데이터	<code>cost_forecast.actual_cost</code>
"프로젝트별"	부서/프로젝트 조인	<code>JOIN departments USING(dept_id)</code>
"거의 안 쓰는 서버"	낮은 CPU/MEM 사용률	<code>cpu_util_pct < 20 AND mem < 30</code>
"미해결 장애"	Open 또는 In Progress	<code>status IN ('Open', 'In-Progress')</code>

Semantic Layer가 없으면, LLM은 매번 "이번 달"을 다르게 해석한다!

가짜 SQL은 DB에 닿기 전에 잡는다

🛡️ SQL GUARD — VALIDATE_SQL()

✅ SELECT / WITH만 허용

DB 변경 가능 모든 키워드 거절

❌ DROP · DELETE · UPDATE · INSERT · ALTER 차단

파괴적 쿼리는 LLM이 생성해도 통과 못함

📋 FROM / JOIN에 실제 테이블 필수

SELECT '보고서...' 같은 가짜 결과 차단

🔒 역할별 테이블 권한 확인

권한 없으면 DB 조회 전에 즉시 중단

👤 개인정보 마스킹

비권한 역할이면 이름, 이메일 자동 가림

👥 RBAC — 역할별 권한

역할	조회 가능 데이터	비용
IT 관리자	전체 + 감사 + AI 비용	✅
인프라 엔지니어	인프라·장애·변경·클라우드 비용	✅
데이터 분석가	장애·보안·권한·KPI	❌
보안 운영	보안·장애·권한 요청·직원	❌
아키텍트	기술검토·장애·인프라·부서	❌
IT 직원	장애·기술검토 (기본)	❌

AI 비용은 측정하지 않으면 폭주한다

ONDOL은 "어떤 모델이 얼마를 썼는지" 모든 호출을 기록한다. 운영자는 이것을 화면에서 본다.

DEFAULT MODEL

gpt-4.1-nano

호출당 평균 ~\$0.0002

PRESET SQL

LLM 0회

자주 쓰는 리포트는 템플릿으로

TRACKED LOGS

3개

api cost · audit · conversation

모든 호출에 남는 것

MODEL 어떤 모델로 호출했는가	TOKENS 입력/출력 토큰 수
COST USD 비용 (자동 환산)	FEATURE SQL 생성 / 리포트 작성 등
ROLE 어떤 역할이 호출했는가	TIMESTAMP 일자별 추이 분석용

감사 로그 — AUDIT_LOG

LOGIN	data@ondol.demo · Data Analyst · 14:23:11
QUERY	"이번 달 클라우드 비용을 프로젝트별로..."
BLOCK	cost_forecast — RBAC denied for role=data_analyst
QUERY	"미해결 장애가 가장 많은 팀은?"
REPORT	8 rows returned · gpt-4.1-nano · \$0.0003

[DEMO] 채팅 한 줄이 리포트가 된다

사용자가 보는 것은 자연어 질문창, 처리 단계, 그리고 자연어 리포트. SQL은 접힌 영역서 확인 가능!

채팅 화면 (메인)

채팅 현황판 AI 사용량 기록 DB/권한

ONDOL 채팅 현황판 AI 사용량 기록 DB/권한 역할: IT 관리자

+ 새 대화

실시간 IT KPI

18 진행 중 P1	129 접근 대기
275 열린 알림	82 작업 위 VM

내가 볼 수 있는 범위

IT 관리자
모든 데이터, 작업 기록, AI 사용량 전체 보기
볼 수 있는 데이터
incidents access_requests arb_reviews
infra_assets security_alerts employees
departments change_log cost_forecast
kpi_snapshots

사용 가능한 도우미
데이터 조회와 리포트

바로 써볼 질문
데이터 조회와 리포트로 해볼 일
> 이번 달 클라우드 비용을 프로젝트별로 리포트로 정리해줘
> 최근 위험률은 어떻게 돼? 의미와 원인을 쉽게 설명해줘
> 이번 달 가장 심각한 장애는 몇 건이고 어느

IT 데이터를 쉽게 물어보세요
질문을 SQL로 바꿔 데이터를 조회하고, 결과를 자연어 리포트로 정리합니다.

IT 관리자 · 모든 데이터, 작업 기록, AI 사용량 전체 보기

이번 달 클라우드 비용을 프로젝트별로 리포트로 정리해줘

Enter 전송 · Shift+Enter 줄바꿈 · 역할별 권한 적용 · 개인정보 자동 가림 · 기록 저장

이 화면에서 보여줄 것

1 추천 질문 클릭

역할별로 다른 추천 표시

2 실시간 처리 단계

구조 확인 → 의미 → SQL → 조회 → 리포트

3 컬러 SQL 코드 블록

생성된 SQL을 하이라이트로 표시

4 자연어 리포트 (메인)

요약 · 해석 · 다음 확인 포인트

5 근거 데이터 + SQL 접힘

표 · CSV 다운로드 · 간단 차트

6 호출 비용 카드 (관리자)

이번 답변에 든 비용을 즉시 표시

예시 질문:

이번 달 클라우드 비용을 프로젝트별로

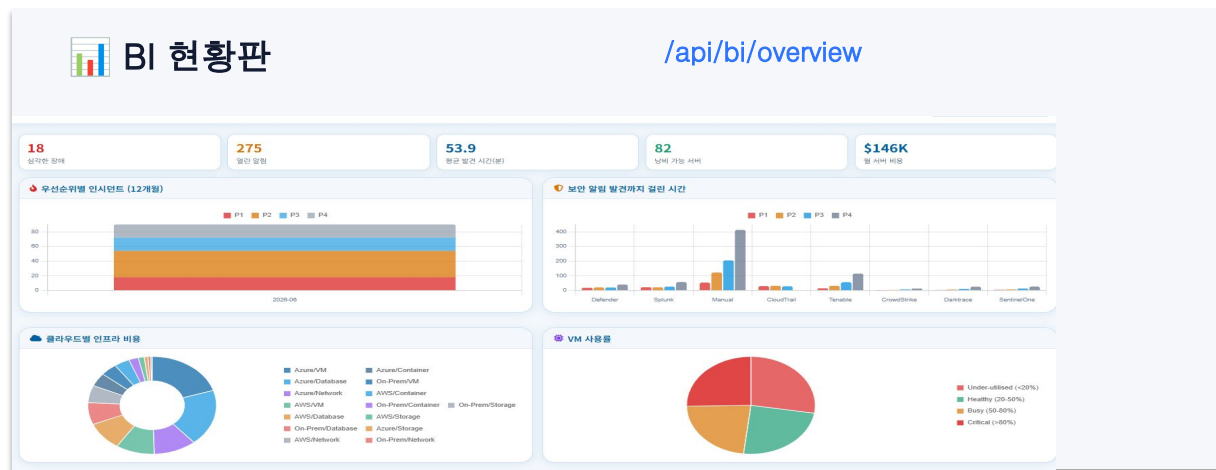
최근 위험률, 의미와 원인 설명

미해결 장애가 가장 많은 팀

거의 안 쓰는데 비용 나가는 서버

[DEMO] 투명하게 모두 보인다!

IT 관리자는 4가지 운영 화면을 본다 — 비용, 감사, DB/권한, BI 현황판. 모두 같은 사이드바.



\$ AI 사용량 [api_cost_log](#)

231 AI 사용 횟수, \$0.1665 총 비용, \$0.00072 1회 평균 비용, 729,053 처리한 문서량

기능별 AI 사용량

모델	작업	횟수	비용	처리량	단위당 비용
gpt-4.1-nano	arch_review	10	6,250	6,789	\$0.00354
gpt-4o-mini	evaluator	9	8,918	1,631	\$0.00232
gpt-4.1-nano	supervisor	12	8,129	1,541	\$0.00143
gpt-4.1-nano	text_to_sql_retry	2	18,197	589	\$0.00126
gpt-4o-mini	schema_discovery	4	4,197	623	\$0.00180
gpt-4.1-nano	infra_ops	4	2,189	1,543	\$0.00084
gpt-4.1-nano	security_triage	2	1,636	661	\$0.00043
gpt-4o-mini	security_triage	1	783	362	\$0.00034

최근 AI 사용 기록

일자	사용 횟수	입력	출력	비용	시간
2024-06-28	231	-	-	\$0.16658	-

기록 / 감사 로그 [audit_log + conversation_log](#)

작업 기록

작업	사용자	역할	성세	시간
PIPELINE	U001	IT Admin	agents=[text_to_sql] eval=100 cost=\$0.00071 retried=False ms=6534	20:06:22
LOGIN	U001	User Alex (IT Admin)	logged in	19:53:55
LOGOUT	U003	Security Ops	세션 종료	19:53:55
PIPELINE	U003	Security Ops	agents=[text_to_sql] eval=100 cost=\$0.00164 retried=False ms=7463	19:45:12
LOGIN	U003	Security Ops	User Casey (Security Ops) logged in	19:44:55
LOGOUT	U002	Architect	세션 종료	19:44:55
LOGIN	U002	Architect	User Blair (Architect) logged in	19:44:52
LOGOUT	U001	IT Admin	세션 종료	19:44:52
PIPELINE	U001	IT Admin	agents=[text_to_sql] eval=100 cost=\$0.00160 retried=False ms=5131	19:44:31

대화 기록 (개인정보 가림)

순서	화자	기능	내용	보관 기한
1	assistant	text_to_sql	이번 달 클라우드 비용을 프로젝트별로 정리해드리겠습니다. 전체 비용 중 가장 큰 항목은 'IT Infrastructure' 프로젝트의	2027-06-28
1	user	stream	이번 달 클라우드 비용을 프로젝트별로 리포트로 정리해줘	2027-06-28
1	assistant	text_to_sql	이번 달에 발생한 보안 취약점(보안 알림) 현황을 정리해드리겠습니다. 전체 120건의 보안 알림이 보고되었으며, 이 중 일부는 아직 해	2027-06-28
1	user	stream	보안 취약점 이번달	2027-06-28

DB / 권한 [/api/schema + /api/db/preview](#)

역할별 데이터 권한

역할	핵심 권한	조회 가능한 테이블
IT 관리자 IT Admin	관리자, 비용 조회	incidents, access_requests, arb_reviews, infra_assets, security_alerts, employees, departments, change_log, cost_forecast, kpi_snapshots, api_cost_log, audit_log, conversation_log
아키텍트 Architect	비용 제한	arb_reviews, incidents, infra_assets, departments
보안 운영 Security Ops	비용 제한	security_alerts, incidents, access_requests, employees
인프라 엔지니어 Infra Engineer	비용 조회	infra_assets, incidents, departments, cost_forecast, change_log
데이터 분석가 Data Analyst	비용 제한	incidents, security_alerts, access_requests, arb_reviews, infra_assets, departments, kpi_snapshots

DB 테이블 현황

테이블	행 수	컬럼	주요 컬럼
access_requests	780	13	req_id, requestor_id, target_system, access_level, status, submitted_at, decided_at, sla_hours, dept_id, risk_level, jit_access, expiry_days, justifica...

[DEMO] RBAC 작동하는 순간

같은 질문을 두 번 한다. 한 번은 IT 관리자로, 한 번은 데이터 분석가로. 결과가 다르다.

QUESTION "이번 달 클라우드 비용을 프로젝트별로 리포트로 정리해 줘"

A CASE A

✓
SUCCESS

admin@ondol.demo · IT 관리자

- ① Schema Discovery — cost_forecast, departments
- ② Semantic — 이번 달 = MAX(month), 프로젝트별 = JOIN dept
- ② SQL Generator — SELECT GROUP BY 생성
- ✓ RBAC 통과 — cost_forecast 접근 권한 있음
- ⑤ DB 조회 — 12 rows × 4 cols
- ⑥ 자연어 리포트 작성 완료
총 \$184,200 · Insurance Core 38% · 다음 확인...

B CASE B

→
BLOCKED

data@ondol.demo · 데이터 분석가

- ① Schema Discovery — cost_forecast 식별
- ② Semantic — 동일 해석 적용
- ② SQL Generator — 동일 SQL 생성

⊘ RBAC 차단 — 실행 전 중단
DENIED role=data_analyst
table=cost_forecast not in allowed_tables
→ audit_log INSERT (block 이벤트 기록)

ONDOL이 가는 곳

NOW — HACKATHON

Stage 1 Lightweight Demo

- ✓ SQLite + Flask
- ✓ gpt-4.1-nano + 프리셋
- ✓ 6개 역할 RBAC
- ✓ Semantic Layer

Q3 2026

Stage 2 Enterprise DW 연결

- MetLife Databricks 어댑터
- 사내 SSO + LDAP RBAC 연동
- Semantic Layer 확장
- 부서별 추천 질문 큐레이션
- 사내 BI 도구 임베드

Q4 2026

Stage 3 운영 강화

- 멀티 모델 라우팅 (난이도별)
- 자동 회귀 평가 (SQL 정확도)
- 알림 / 스케줄 리포트
- 외부 감사 보고서 자동 생성
- Teams 채널 연동 등 (Copilot Studio 등 활용)

VISION · 2027

Stage 4 → AI Platform과의 통합

- LLM 대신 sLLM 등 사용
- 클레임·언더라이팅 등 타 도메인으로 확장 (->현업)
- AI 통제 구조를 사내 표준으로

SQL을 대신 써주는 도구가 아니라, 의사결정을 돕는 데이터 에이전트.

ONDOL

01

사용자에겐 자연어 리포트

SQL을 모르고도 데이터에 접근. 결과는 표가 아니라 의미가 담긴 글.

02

시스템엔 강력한 통제

환각·권한·근거·설명·비용 — 모두 강제. "감사받을 수 있는" SQL.

03

미래는 MetIQ로 통합

독립 서비스가 아니라 MetLife AI 플랫폼의 안전 설계 모듈로 합류. (그러나 Databricks genie AI같은 솔루션도 좋은 대안!)

DEMO

 **ONDOL**
MetLife AI WOW · IT 플랫폼 데모

이메일

admin@ondol.demo

비밀번호

로그인

- 데모 역할 빠른 로그인 -

A IT 관리자 admin	R 아키텍트 arch	S 보안 운영 sec
I 인프라 엔지니어 infra	D 데이터 분석가 data	U IT 직원 staff

채팅 한 줄이 리포트가 된다

사용자가 보는 것은 자연어 질문장, 처리 단계, 그리고 자연어 리포트. SQL을 접한 영역에서 확립 가능!

특정 부서 (선택)

부서

전체

인사

영업

기타

관리자

채팅

이력

리포트

설정

1. 주된 질문 등록

자연어로 등록하는 질문

2. 질문의 처리 단계

이름: 채팅 → 처리 → SQL → 결과 → 리포트

3. 필터링 SQL, 쿼리 등록

필터링 쿼리를 입력하여 쿼리를 등록

4. 자연어 리포트 (선택)

이름: 채팅, 리포트, 쿼리, 결과

5. 결과 처리 및 SQL 등록

이름: 채팅, 리포트, 쿼리, 결과

6. 최종 리포트 처리 (선택)

이름: 채팅, 리포트, 쿼리, 결과

Appendix.

(1) 질문 이해 → SQL 생성·검증

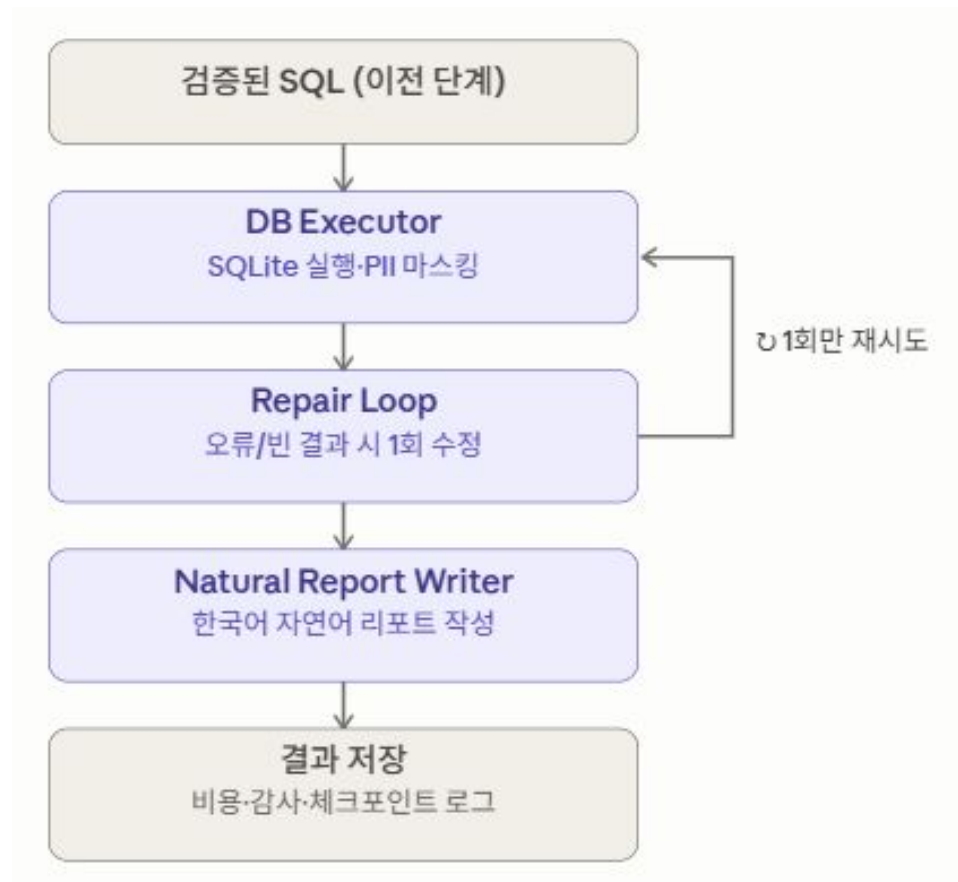
- 보라색은 LLM 에이전트가 처리하는 단계, 회색은 진입점·분기 종료점 .
- Router(high-cost LLM) 에서 데이터 질문이 아니면 SQL을 만들지 않고 바로 종료하고, RBAC Guard에서도 권한이 없으면 DB 실행 전에 차단.
- 검증을 통과한 SQL만 다음 단계로 넘어감.
- 여기서 Router는 가장 핵심적인 역할인 의도 분류를 하기 때문에 가장 고성능의 LLM을 사용.



Appendix.

(2) DB 실행 → 리포트 작성 → 저장

- 검증된 SQL 실행
- 실행 후 오류나 빈 결과가 나왔을 때 1회만 재시도하는 Repair Loop가 포함된다.

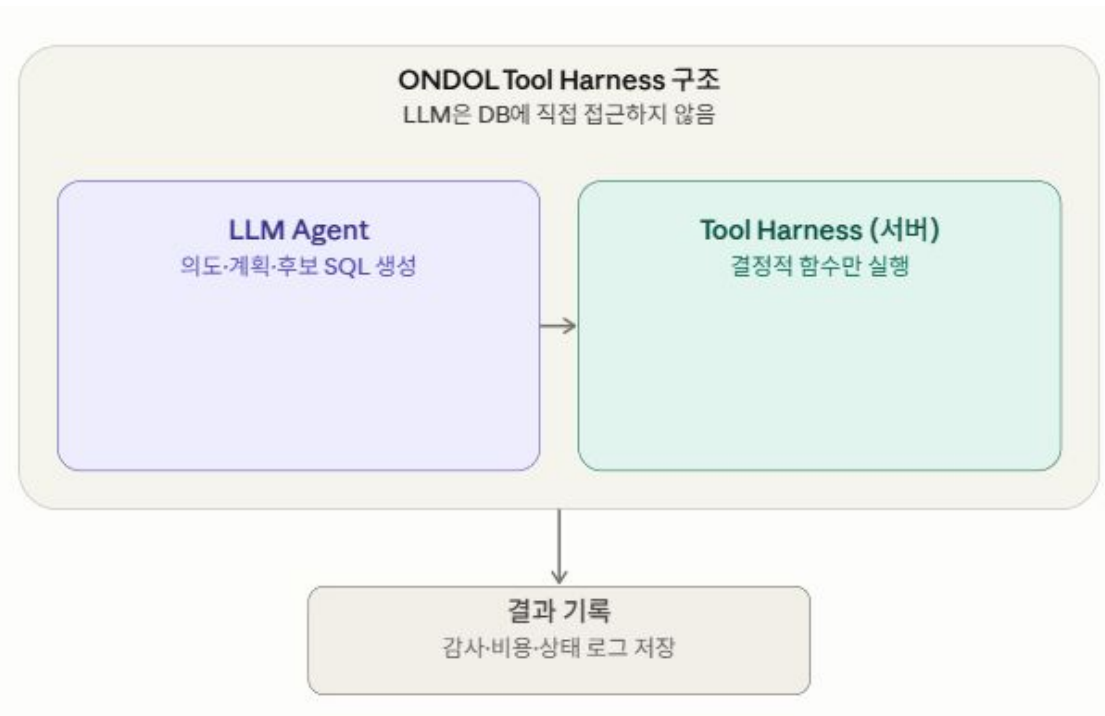


Appendix.

(3) ONDOL Tool Harness

- LLM 에이전트는 SQL이나 결과를 직접 만지지 않고, "의도·계획·후보 SQL·리포트 초안" 같은 구조화된 출력만 내놓음
- 실제 DB 접근이나 검증처럼 부작용이 있는 작업은 전부 서버 쪽 결정적 함수가 대신 실행함.
- Tool Harness에 등록된 실제 함수는 6개:
 - `validate_sql` (`SELECT-only`·위험 키워드·권한 검증)
 - `execute_sql` (검증된 SQL만 실행),
 - `sql_repair` (1회 한정 수정)
 - `mask_result_pii` (개인정보 마스킹)
 - `log_api_cost` (토큰·비용 기록)
 - `checkpoint_state` (AgentState 단계별 저장)

이 구조 덕분에 LLM의 권한이 항상 서버 코드로 제한된다.



Appendix.

(4) Multi-Agent Orchestration

- **Router** - 사용자의 의도 분류 / 에이전트 호출 결정
- **Schema Discovery** - Text-to-SQL이 실행되기 전, 접근 가능한 테이블·컬럼 구조 확인 -> 컨텍스트로 전달
- **Query Planner** - 복잡한 질문을 사전에 분해·계획하는 역할. 결과를 Text-to-SQL에게 전달
- **Text-to-SQL** - 핵심 Agent. validate_sql로 구문·권한을 검사하고, execute_sql로 실제 SQLite 쿼리를 실행. 오류 시 sql_repair로 1회 재시도.
- **SQL Critic** - (비평가) Text-to-SQL이 만든 SQL을 독립적으로 재검토·교정. validate_sql을 직접 호출해 이중 검증.

